Amazon Web Services

Networking & Content Delivery Complete Learning Guide

Introduction | Core Concepts | Architecture | Features | Use Cases | Integrations | Comparisons

Beginner-Friendly • In-Depth • Practical

Table of Contents

Each chapter covers: Simple Introduction • Real-World Analogy • Core Concepts & Terminology • Architecture & Workflow • Key Features • Real-World Use Cases (by industry) • AWS Service Integrations • Comparison Tables.

Chapter 1 — Amazon CloudFront

Global Content Delivery Network (CDN) with 400+ Edge Locations

1. Simple Introduction

Amazon CloudFront is a fast, global Content Delivery Network (CDN) service that securely delivers data, videos, applications, and APIs to users worldwide with low latency and high transfer speeds. It does this by caching content at edge locations that are physically close to end users — so instead of requests travelling to a server in another continent, they are served from a nearby city.

A CDN (Content Delivery Network) is a globally distributed network of servers. When a user requests a file (image, video, webpage), the CDN serves it from the nearest server rather than the origin server, dramatically reducing load time.

Analogy — The Neighbourhood Grocery Store

Imagine your origin server is a giant central warehouse in Mumbai. Without CloudFront, every customer in New York, London, and Tokyo has to wait for delivery from Mumbai. CloudFront is like opening local grocery stores (edge locations) in every major city. Popular items (content) are stocked locally. Customers get instant service. The Mumbai warehouse only ships new stock when items run out (cache miss).

Business Problem CloudFront Solves

Global users experience high latency fetching content from a single origin region

Origin servers get overwhelmed by repeated requests for the same static files

DDoS attacks can take down origin servers — CloudFront absorbs and filters attacks

Delivering HD video, large files, or APIs at scale without huge infrastructure cost

Need HTTPS/TLS everywhere, including custom SSL certificates and modern protocols

2. Core Concepts & Terminology

Term	Definition	Example
CDN (Content Delivery Network)	A distributed system of servers that caches and delivers content to users from geographically close locations	CloudFront, Akamai, Cloudflare
Edge Location	A CloudFront data centre (PoP — Point of Presence) that caches content close to users. 400+ worldwide	Mumbai, London, São Paulo edge
Regional Edge Cache	Larger mid-tier caches between origin and edge locations — hold less-popular content longer	Reduces origin load for moderate-traffic content
Distribution	A CloudFront configuration that defines how content is delivered (origins, cache behaviours, security)	One distribution per website/API
Origin	The source of truth for content — CloudFront fetches from here on a cache miss	S3 bucket, ALB, EC2, API Gateway, custom server
Cache Behaviour	A set of rules (path pattern + settings) applied to requests matching that pattern	/images/* caches for 24h; /api/* never caches

Term	Definition	Example
Cache Hit	Request served directly from edge location — no call to origin; fastest response	Edge returns cached index.html
Cache Miss	Content not in edge cache — CloudFront fetches from origin, caches it, returns to user	First request for a new image
TTL (Time To Live)	How long an object stays in the edge cache before CloudFront re-fetches from origin	Default 24 hours; 0 for dynamic content
Invalidation	Force CloudFront to delete cached objects before TTL expires	After deploying new JS/CSS files
Signed URL	A time-limited, user-specific URL granting access to a private object	Premium video download link valid for 1 hour
Signed Cookie	Like Signed URL but grants access to multiple files via a cookie	Authenticated user accessing a private video library
Origin Access Control (OAC)	Restricts S3 bucket to only allow CloudFront to access it — blocks direct S3 URL access	S3 website only accessible via CloudFront
Lambda@Edge	Run Lambda functions at CloudFront edge locations to transform requests/responses	A/B testing, auth checks, URL rewriting at edge
CloudFront Functions	Lightweight JavaScript functions running at edge — faster and cheaper than Lambda@Edge	HTTP header manipulation, URL redirects
WAF (Web Application Firewall)	Filter malicious web traffic (SQL injection, XSS, bots) before it reaches your origin	Block requests from known bad IPs
Price Class	Choose which regions to serve from based on cost vs latency trade-off	Price Class 100 = US/EU/Asia only

3. Architecture & How CloudFront Works

CloudFront Request Flow (Step by Step)

USER in Tokyo requests: https://d1234.cloudfront.net/hero-image.jpg

STEP 1 — DNS Resolution

CloudFront DNS returns IP of nearest edge location (e.g. Tokyo PoP)

STEP 2 — Edge Location Check

Tokyo edge looks up hero-image.jpg in its cache

CACHE HIT -> Return image immediately (sub-10ms response) [DONE]

CACHE MISS -> Continue to Step 3

STEP 3 — Regional Edge Cache Check

Tokyo edge queries nearest Regional Edge Cache (e.g. Singapore REC)

HIT -> REC returns image, edge caches it, returns to user [DONE]

MISS -> Continue to Step 4

STEP 4 — Origin Fetch

CloudFront connects to Origin (e.g. S3 in us-east-1) over AWS backbone

Fetches hero-image.jpg from S3

Caches in Regional Edge Cache (Singapore) and Tokyo edge

Returns image to user in Tokyo

ALL FUTURE Tokyo requests -> CACHE HIT -> served instantly

SECURITY LAYERS APPLIED AT EDGE:

- TLS/HTTPS termination at edge (user <-> edge always HTTPS)
- AWS WAF rules evaluated (block SQL injection, bad bots)
- AWS Shield Standard DDoS protection always active
- Lambda@Edge / CloudFront Functions can modify request/response

4. Key Features

Feature	Description	Business Value
400+ Global Edge Locations	PoPs in 90+ cities across 47 countries	Sub-50ms latency for most of the world's population
HTTPS & TLS Everywhere	Free TLS certificate via AWS Certificate Manager; TLS 1.2/1.3; HTTP/2 and HTTP/3 (QUIC)	Security and performance by default
Static & Dynamic Content	Cache static files; proxy dynamic API calls with low-latency AWS backbone routing	One CDN for entire application
Lambda@Edge	Run Node.js/Python Lambda at edge on viewer/origin request/response	Personalisation, auth, A/B testing at edge
CloudFront Functions	Lightweight JS at edge; <1ms execution; 10x cheaper than Lambda@Edge	URL rewrites, header manipulation, redirects
AWS WAF Integration	Block OWASP Top 10 threats, rate-limit, geo-block	Application-layer DDoS and attack protection
AWS Shield Standard	Always-on DDoS protection at no extra cost	Absorb volumetric DDoS attacks at edge

Feature	Description	Business Value
Origin Shield	Extra caching layer in front of origin; reduces origin load	Protect fragile or expensive origins
Origin Failover	Primary + secondary origin; auto-failover on 5xx errors	High availability for dynamic content
Signed URLs / Cookies	Time-limited, user-specific access to private content	Paid video streaming, secure downloads
Geo-Restriction	Allow or block users by country	Content licensing, GDPR compliance, export control
Real-Time Logs	Stream request logs to Kinesis Data Streams in real time	Live traffic analytics and security monitoring
Cache Policies & Origin Request Policies	Fine-grained cache key control (headers, cookies, query strings)	Cache personalised and public content correctly
Continuous Deployment	Test new distribution config on a subset of traffic before full rollout	Safe CDN config changes

5. Real-World Use Cases

Industry	Use Case	CloudFront Feature Used
E-Commerce (Amazon-scale)	Cache product images, CSS, JS globally; users in every country see fast page loads	Edge caching + S3 origin + OAC
Video Streaming (Netflix-style)	Deliver HLS/DASH video segments from S3 to millions of concurrent viewers globally	Signed URLs + Range GETs + Origin Shield
Gaming	Distribute game patches, assets, and updates globally with low latency	Large file delivery + edge caching
Banking / Fintech	Serve static web app via CloudFront; API calls to ALB proxied with WAF protection	WAF + HTTPS + Lambda@Edge auth
Healthcare (HIPAA)	Deliver patient portal frontend; WAF blocks malicious traffic; geo-restrict to permitted regions	WAF + geo-restriction + signed cookies
News / Media	Handle viral traffic spikes — millions of requests/min for breaking news articles	Edge caching absorbs origin load
SaaS (Multi-Tenant)	Route requests to tenant-specific origins based on subdomain using Lambda@Edge	Lambda@Edge origin routing
API Acceleration	Proxy API calls through CloudFront to reduce latency via AWS backbone	Dynamic content + compression

6. AWS Service Integrations

AWS Service	Integration	Use Case
Amazon S3	S3 as origin with OAC — only CloudFront can access bucket	Static website / file hosting
AWS ALB / EC2	ALB or EC2 as origin for dynamic content	Full-stack web application delivery
API Gateway	CloudFront in front of API Gateway for global API acceleration	Low-latency REST / HTTP APIs worldwide
AWS WAF	Attach WAF WebACL to CloudFront distribution	Block OWASP threats and bots at edge
AWS Shield Advanced	Upgrade DDoS protection with 24x7 DRT support	Enterprise DDoS mitigation

AWS Service	Integration	Use Case
AWS Certificate Manager (ACM)	Free SSL/TLS certificates for custom domains	HTTPS on custom domain at no cost
Lambda@Edge	Trigger Lambda on CloudFront events (4 hook points)	Edge compute: auth, personalisation, rewriting
Amazon Kinesis Data Streams	Real-time log delivery for analytics	Live security monitoring and analytics
Amazon CloudWatch	Metrics: requests, cache hit ratio, error rates, latency	Performance monitoring and alerting
AWS Route 53	Alias records pointing to CloudFront distribution domain	Custom domain (www.example.com) for CDN

7. CloudFront vs Other CDN Solutions

Feature	CloudFront	Cloudflare	Akamai	Azure CDN
AWS Integration	Native / Deep	Manual	Manual	Azure-native
Edge Locations	400+	200+	4,000+	130+
WAF	AWS WAF (paid add-on)	Included free tier	Enterprise only	Azure WAF
Edge Compute	Lambda@Edge + CF Functions	Workers	EdgeWorkers	Azure Functions
DDoS Protection	Shield Std (free)	Free tier	Enterprise	Azure DDoS
Pricing	Pay per GB + request	Free tier + paid	Enterprise contract	Pay per GB
Private Content	Signed URL/Cookie	Signed URL	Token Auth	SAS token
Best For	AWS workloads, S3/ALB origins	General web, DNS-first	Enterprise telco/media	Azure workloads

Chapter 2 — AWS PrivateLink

Private Connectivity Between VPCs and AWS Services Without the Internet

1. Simple Introduction

AWS PrivateLink is a networking technology that allows you to access AWS services and services hosted by other AWS customers (or partners) privately — without exposing traffic to the public internet. Traffic between your VPC and the service travels entirely within the AWS network using private IP addresses.

Before PrivateLink, if you wanted to access an AWS service (like S3 or DynamoDB) from within a VPC, the traffic either had to leave your VPC to the public internet, or you had to set up complex VPC peering and routing. PrivateLink creates a direct private tunnel into the service using a VPC Endpoint.

Analogy — The Private Office Corridor

Imagine Company A (your VPC) and Company B (a SaaS service) share the same office building (AWS network). Without PrivateLink, data has to leave the building, go down the public street, and re-enter Company B's entrance — exposed to traffic. PrivateLink builds a private internal corridor connecting both companies' offices directly — no public street, no exposure, no risk.

Business Problem PrivateLink Solves

Access AWS services (S3, DynamoDB, SageMaker) from a VPC without internet gateway

Expose your SaaS product to customer VPCs without VPC peering or IP conflicts

Comply with regulations requiring all data to stay within the private network

Prevent data exfiltration — if there's no internet route, data cannot leave

Multi-tenant SaaS: each customer connects privately to your service endpoint

2. Core Concepts & Terminology

Term	Definition	Example
VPC Endpoint	A virtual device in your VPC that enables private connectivity to AWS services or PrivateLink services without internet gateway	Endpoint for S3 in your VPC
Interface Endpoint	A network interface (ENI) with a private IP in your subnet — used for most AWS services and PrivateLink. Powered by PrivateLink	Private IP for SSM, Secrets Manager, ECR
Gateway Endpoint	A special gateway (not PrivateLink) for S3 and DynamoDB only — free, route-table based	Add S3 gateway endpoint to route table
Gateway Load Balancer Endpoint	PrivateLink endpoint that routes traffic to third-party virtual appliances (firewalls, IDS)	Palo Alto firewall as a PrivateLink service
Endpoint Service	A service you create and expose to other VPCs or accounts via PrivateLink (you are the provider)	Your NLB-backed API exposed to customers

Term	Definition	Example
Service Provider	The AWS account hosting the NLB-backed service that is exposed via PrivateLink	SaaS company exposing their API
Service Consumer	The AWS account creating an Interface Endpoint to consume the provider's service	Customer VPC connecting to SaaS
ENI (Elastic Network Interface)	A virtual network card — Interface Endpoints are ENIs with private IPs in your subnets	Private IP 10.0.1.45 in your subnet
NLB (Network Load Balancer)	The load balancer that fronts your service in a PrivateLink Endpoint Service setup	Distributes consumer traffic to your service fleet
Endpoint Policy	IAM resource-based policy on a VPC Endpoint controlling which principals can use it and which actions/resources are allowed	Allow only specific S3 bucket via endpoint
Private DNS	Resolve standard AWS service DNS names (e.g. s3.amazonaws.com) to private endpoint IPs instead of public IPs	s3.amazonaws.com -> 10.0.1.45

Endpoint Types — Quick Comparison

Type	Technology	Services	Cost	Setup
Interface Endpoint	PrivateLink (ENI in subnet)	Most AWS services + custom	~$0.01/hr per AZ + data	Security Group + DNS
Gateway Endpoint	Route-table prefix list	S3 and DynamoDB only	Free	Route table entry
GatewayLB Endpoint	PrivateLink + GWLB	3rd-party virtual appliances	~$0.01/hr + data	Route table entry

3. Architecture & How PrivateLink Works

Consumer-to-AWS-Service Architecture

PrivateLink — Accessing AWS Services Privately

YOUR VPC (10.0.0.0/16)

|

| [EC2 / Lambda] -- no internet gateway needed --

| |

| [Interface Endpoint ENI] (Private IP: 10.0.1.55)

| | Private DNS: secretsmanager.us-east-1.amazonaws.com -> 10.0.1.55

| | (all traffic stays inside AWS network)

| |

+-------+-----> [AWS PrivateLink backbone]

|

[AWS Secrets Manager Service]

(AWS-managed VPC, not your internet)

Result: No Internet Gateway, No NAT Gateway, No public IP needed.

Traffic NEVER leaves the AWS private network.

Provider-Consumer Architecture (Custom PrivateLink Service)

SaaS Provider exposing service via PrivateLink

PROVIDER ACCOUNT (SaaS Company)

[Your Service Servers (EC2/ECS/EKS)]

|

[Network Load Balancer (NLB)] <- PrivateLink requires NLB

|

[Endpoint Service] <- You create this; share service name

| (service name: com.amazonaws.vpce.us-east-1.vpce-svc-0abc123)

AWS PrivateLink backbone

|

CONSUMER ACCOUNT (Customer VPC)

[Interface Endpoint ENI] (Private IP in customer subnet)

|

[Customer's EC2 / Lambda] -- calls endpoint private IP --

KEY PROPERTIES:

- Customer VPC and Provider VPC never peer / never share address space
- Overlapping CIDRs (10.0.0.0/16 on both sides) is NOT a problem
- Provider can whitelist/accept specific consumer accounts
- Traffic is unidirectional: consumer -> provider only

4. Key Features

Feature	Description	Business Value
No Internet Exposure	Traffic never traverses public internet — stays on AWS backbone	Eliminates internet-based attack surface
No VPC Peering Required	Consumer and provider VPCs stay completely separate	Works even with overlapping IP ranges
Overlapping CIDR Support	PrivateLink works even when both VPCs have same IP ranges	Simplifies multi-tenant SaaS architectures
Endpoint Policies	IAM-like policies on endpoints restrict which resources are accessible	Fine-grained access control at network layer
Private DNS Integration	AWS service hostnames resolve to private endpoint IPs automatically	Zero application code changes
Cross-Account & Cross-Region	Share services across AWS accounts and (with Transit GW) regions	Enterprise multi-account connectivity
Scalable to Thousands	Thousands of consumers can connect to one Endpoint Service	Scalable SaaS distribution model
Security Group Support	Interface endpoints respect Security Groups for access control	Network-layer allow/deny rules
Availability	Endpoints deployed per-AZ for high availability	No single point of failure
Audit & Logging	VPC Flow Logs capture endpoint traffic; CloudTrail logs API calls	Full audit trail for compliance

5. Real-World Use Cases

Scenario	How PrivateLink Is Used	Benefit
Regulated Industries (Banking/Healthcare)	EC2 in private subnets accesses Secrets Manager, S3, and DynamoDB via Interface Endpoints — zero internet traffic	HIPAA / PCI-DSS compliance — no internet egress
SaaS Distribution	SaaS vendor exposes their API as an Endpoint Service; customers connect from their VPCs privately	No IP peering, works across overlapping CIDRs
Data Exfiltration Prevention	Endpoint policies restrict S3 access to only the company's own buckets — even if credentials are stolen	Data cannot be copied to external S3 buckets
Multi-Account Architecture	Centralise shared services (logging, security tools) in one account; all others connect via PrivateLink	Clean account isolation without complex routing
Partner API Access	A financial data provider exposes market data API to bank customers via PrivateLink	Banks never expose their VPC to the internet
On-Premises to AWS Services	On-prem servers use Direct Connect + PrivateLink to access AWS services privately	Private access from data centre without internet
Marketplace Vendors	AWS Marketplace SaaS products use PrivateLink for private delivery to buyer VPCs	Secure product delivery at scale

6. AWS Service Integrations

AWS Service	Integration	Use Case
Amazon VPC	Interface Endpoints live inside VPC subnets as ENIs	Foundation — PrivateLink is a VPC feature
AWS Network Load Balancer	NLB is required to front your service in an Endpoint Service	Provider-side load balancing
AWS Direct Connect	On-premises access to Interface Endpoints via DX private VIF	Private access from data centre
AWS VPN	Access Interface Endpoints via Site-to-Site VPN	VPN-connected networks access AWS services privately
AWS Transit Gateway	Route traffic from many VPCs to shared PrivateLink endpoints	Centralised endpoint sharing across many VPCs
AWS IAM	Endpoint policies use IAM syntax for resource-level access control	Fine-grained access control on endpoints
VPC Flow Logs	Log all traffic through endpoints for auditing	Compliance and security monitoring
AWS CloudTrail	Audit API calls to endpoint service management APIs	Change management audit trail
AWS Marketplace	Marketplace SaaS products delivered via PrivateLink	Private SaaS product access

7. PrivateLink vs VPC Peering vs VPN vs Transit Gateway

Feature	PrivateLink	VPC Peering	Site-to-Site VPN	Transit Gateway
Purpose	Expose specific service privately	Connect two VPCs fully	Connect on-prem to VPC	Hub for many VPCs/VPNs
Overlapping CIDRs	Supported	Not supported	Depends	Not supported

Feature	PrivateLink	VPC Peering	Site-to-Site VPN	Transit Gateway
Traffic Direction	One-way (consumer to service)	Bidirectional	Bidirectional	Bidirectional
Internet Needed	No	No	Uses public internet	No
Scale	1000s of consumers per service	1:1 relationship	Per tunnel limits	5,000 VPCs per TGW
Cross-Account	Yes	Yes	N/A	Yes
Cross-Region	Limited (via TGW)	Yes (peering)	Yes	Yes (peering)
Best For	Service exposure, SaaS, data security	Simple VPC-to-VPC	On-prem to AWS	Many VPCs, complex routing

Chapter 3 — Amazon Route 53

Scalable, Highly Available Cloud DNS Service with Traffic Routing

1. Simple Introduction

Amazon Route 53 is a highly available, scalable, and fully managed Domain Name System (DNS) web service. It is designed to give developers and businesses an extremely reliable and cost-effective way to route end users to internet applications by translating human-readable domain names (like www.example.com) into numeric IP addresses (like 192.0.2.1) that computers use to connect to each other.

DNS (Domain Name System) is the internet's 'phone book'. When you type a web address, DNS translates it into an IP address so your browser knows where to connect. Route 53 goes far beyond basic DNS — it includes health checking, traffic routing policies, and domain registration, making it a complete traffic management solution.

The name 'Route 53' is a reference to port 53, the standard network port used for DNS.

Analogy — The World's Fastest Phone Book

Imagine you want to call 'Pizza Palace'. Instead of knowing their number, you ask the phone book. Route 53 is that phone book — but it's instant, global, and smart. It doesn't just give you one number: it checks which branch is open (health check), gives you the closest one (latency routing), and if one branch closes (failure), it immediately gives you the backup branch's number — all in milliseconds.

Business Problem Route 53 Solves

Translate domain names to IPs — the fundamental internet requirement

Automatically route users to the closest, fastest, or healthiest server

Failover traffic to a backup region when the primary goes down

Register and manage domain names (e.g. buy mycompany.com)

Route traffic based on geography, latency, weighted splits, or health

2. Core Concepts & Terminology

Term	Definition	Example
DNS (Domain Name System)	Global system translating domain names to IP addresses	example.com -> 203.0.113.5
Domain Name	Human-readable address of a website or service	www.amazon.com, api.myapp.io
TLD (Top-Level Domain)	The last part of a domain name — managed by IANA registries	.com, .org, .in, .io, .aws
Hosted Zone	A container for DNS records for a specific domain. Like a DNS database for one domain	Hosted Zone for example.com
Public Hosted Zone	DNS records accessible by anyone on the internet	Routes internet traffic to your website

Term	Definition	Example
Private Hosted Zone	DNS records only resolvable within associated VPCs	api.internal.company.com for internal services
DNS Record	An entry in a hosted zone mapping a name to a value	A, AAAA, CNAME, MX, TXT, NS, SOA, SRV
A Record	Maps a hostname to an IPv4 address	www.example.com -> 203.0.113.5
AAAA Record	Maps a hostname to an IPv6 address	www.example.com -> 2001:db8::1
CNAME Record	Maps a hostname to another hostname (alias). Cannot be used for zone apex	shop.example.com -> example.myshopify.com
Alias Record	Route 53-specific extension of A/AAAA record that points to AWS resources. Works at zone apex. Free queries	example.com -> my-alb.us-east-1.elb .amazonaws.com
MX Record	Mail exchange record — directs email to mail servers	example.com MX -> mail.example.com
TXT Record	Stores arbitrary text — used for domain verification, SPF, DKIM	SPF: 'v=spf1 include:amazonses.com ~all'
NS Record	Name Server record — specifies which servers are authoritative for a domain	ns-1234.awsdns-12.com
SOA Record	Start of Authority — administrative info about the zone	Primary NS, admin email, serial number
Health Check	Route 53 periodically tests endpoint availability; used with routing policies for failover	HTTP check on /health every 30s
TTL (Time To Live)	How long DNS resolvers cache a record before re-querying Route 53	TTL=300 means clients cache for 5 min

Routing Policies

Policy	How It Works	Use Case	Health Check?
Simple	Return one IP (or multiple randomly)	Single server, no special routing needed	No
Weighted	Split traffic by percentage (e.g. 80/20)	A/B testing, gradual migration, blue/green	Yes
Latency-Based	Route to AWS region with lowest latency for user	Multi-region apps — give users the fastest experience	Yes
Failover	Primary endpoint; if unhealthy, route to secondary	Active-passive disaster recovery	Yes (required)
Geolocation	Route based on user's geographic location (country/continent)	Content localization, regulatory compliance, language routing	Yes
Geoproximity	Route based on physical distance with configurable bias shift	Fine-tune traffic distribution between regions	Yes
Multi-Value Answer	Return up to 8 healthy IPs; acts as simple load balancer	Multiple healthy resources with random selection	Yes
IP-Based	Route based on user's IP CIDR range	Route corporate IP range to internal endpoint, ISP-specific routing	No

3. Architecture & DNS Resolution Flow

How Route 53 Resolves a DNS Query

USER types: www.example.com in browser

STEP 1 — Local DNS Cache

Browser checks local cache. If TTL not expired -> use cached IP. [DONE]

Cache miss -> continue

STEP 2 — Recursive Resolver (ISP/Google DNS/8.8.8.8)

User's OS queries local recursive resolver (e.g. Google 8.8.8.8)

Resolver checks its cache -> miss -> continue

STEP 3 — Root Nameserver

Resolver queries root nameserver -> returns TLD (.com) nameserver addresses

STEP 4 — TLD Nameserver (.com)

Resolver queries .com TLD server -> returns Route 53 NS records for example.com

(e.g. ns-123.awsdns-45.com)

STEP 5 — Route 53 Authoritative Nameserver

Resolver queries Route 53 (your hosted zone) for www.example.com

Route 53 evaluates routing policy:

- Simple: return 203.0.113.5
- Latency: check user location -> return closest region IP
- Failover: check health check -> primary healthy? return primary : return secondary

Route 53 returns the answer

STEP 6 — Response chain

Resolver caches result (for TTL seconds) -> returns to browser -> browser connects

Route 53 uses Anycast: 100+ Route 53 nameserver PoPs worldwide answer queries from the nearest location -> sub-10ms DNS resolution globally.

4. Key Features

Feature	Description	Business Value
100% SLA Availability	Route 53 is the only AWS service with a 100% uptime SLA	DNS is always up — your domain always resolves
Global Anycast Network	100+ PoPs answer DNS queries from nearest location globally	Sub-10ms DNS resolution worldwide
Domain Registration	Register and manage domain names directly in Route 53	One-stop DNS + domain management
Health Checks	Monitor endpoint health via HTTP/HTTPS/TCP; trigger routing changes	Automated failover without human intervention
Private DNS for VPC	Create private hosted zones — internal DNS for microservices	Service discovery without internet DNS
8 Routing Policies	Simple, Weighted, Latency, Failover, Geo, Geoproximity, Multi-value, IP-based	Sophisticated traffic management

Feature	Description	Business Value
DNSSEC	Sign DNS responses to prevent DNS spoofing/cache poisoning	DNS security for sensitive domains
Traffic Flow (Visual Editor)	Visual policy editor combining multiple routing policies	Complex routing without code
Resolver (Hybrid DNS)	Forward DNS queries between on-premises and VPC	Unified DNS for hybrid cloud
Resolver DNS Firewall	Block DNS queries to known malicious domains	DNS-layer security for VPC workloads
CIDR-Based Routing	Route based on source IP CIDR ranges	Corporate network routing, ISP differentiation
Alias Records (Free)	Point to AWS resources; no charge per query; supports zone apex	Free and flexible AWS resource routing

5. Real-World Use Cases

Industry / Scenario	Route 53 Configuration	Routing Policy Used
Global Web App	api.myapp.com resolves to us-east-1 for US users, eu-west-1 for European users, ap-south-1 for Indian users	Latency-Based Routing
Disaster Recovery	primary.myapp.com points to active region; health check monitors /health; on failure auto-routes to DR region	Failover Routing + Health Check
A/B Testing / Canary	new-feature.myapp.com: 10% traffic to new version, 90% to stable version; gradually shift weight	Weighted Routing
Content Localisation	example.com serves EN for US users, DE for German users, IN for Indian users — different ALBs per region	Geolocation Routing
Internal Microservices	Private hosted zone: cart.internal, payment.internal, inventory.internal — resolvable only inside VPC	Private Hosted Zone
Hybrid Cloud	On-prem servers resolve AWS internal services; Route 53 Resolver forwards on-prem DNS to Route 53	Route 53 Resolver (Inbound/Outbound)
Banking (Multi-Region Active-Active)	api.bank.com resolves to both us-east-1 and eu-west-1; health checks keep only healthy regions active	Multi-Value + Health Checks
Gradual Migration	Migrating from on-prem to AWS: weight 90% to on-prem, gradually increase AWS weight to 100%	Weighted Routing

6. AWS Service Integrations

AWS Service	Integration	Use Case
Amazon CloudFront	Alias record pointing to CloudFront distribution	Custom domain (example.com) for CDN
AWS ALB / NLB	Alias record pointing to load balancer DNS	Custom domain for load-balanced apps
Amazon S3	Alias record for S3 static website endpoint	Custom domain for S3 static website
Amazon API Gateway	Custom domain mapped to API Gateway stage	api.example.com for REST or HTTP API
AWS Elastic Beanstalk	Alias record to Beanstalk environment URL	Custom domain for Beanstalk app
Amazon VPC	Private Hosted Zones associated with VPC for internal DNS	Internal service discovery

AWS Service	Integration	Use Case
Route 53 Resolver	Bidirectional DNS forwarding between VPC and on-premises	Hybrid cloud unified DNS
AWS Health / CloudWatch	Health check alarms integrated with failover routing	Automated traffic failover on alarm
ACM (Certificate Manager)	Domain validation via Route 53 TXT/CNAME records (auto-managed)	One-click HTTPS certificate issuance
AWS Global Accelerator	Use Route 53 to point to GA endpoints for Anycast routing	TCP/UDP performance optimisation + routing

7. Route 53 vs Other DNS Services

Feature	Route 53	GoDaddy DNS	Cloudflare DNS	Azure DNS
SLA	100%	99.9%	100%	100%
Global PoPs	100+	Limited	200+	Limited
Routing Policies	8 types	Basic	Load balancing (paid)	Traffic Manager
Health Checks	Built-in	No	Yes (paid)	Yes (Traffic Manager)
Private DNS	Yes (VPC)	No	No	Yes (Azure VNet)
DNSSEC	Yes	Yes	Yes	Yes
Domain Reg.	Yes	Yes	Yes	No
AWS Integration	Native / Alias	Manual	Manual	Limited
Hybrid DNS	Resolver	No	No	Azure DNS Resolver
Price	$0.50/zone/mo + $0.40/M queries	Included w/ domain	Free + paid	$0.50/zone/mo

Chapter 4 — Amazon Virtual Private Cloud (VPC)

Your Isolated, Logically Defined Private Network Inside AWS

1. Simple Introduction

Amazon VPC (Virtual Private Cloud) is a logically isolated section of the AWS Cloud where you can launch AWS resources in a virtual network that you define. You have complete control over your virtual networking environment — including selecting your own IP address ranges, creating subnets, configuring route tables, and setting up network gateways and security controls.

Every AWS resource (EC2, RDS, Lambda in VPCs, EKS nodes, etc.) lives inside a VPC. Understanding VPC is the foundation of all AWS networking — it is not optional. A default VPC is automatically created in every AWS account in every region, but production workloads always use custom VPCs for security and control.

Analogy — Your Private Office Building

AWS is like a massive city with thousands of buildings (services). Your VPC is your own private office building inside that city. You decide the floor plan (subnets), who has keys (security groups), which doors connect to the street (internet gateway), which rooms are private (private subnets), and which hallways connect floors (route tables). Other tenants in the city cannot walk into your building — it is completely isolated.

Business Problem VPC Solves

Launch AWS resources in a private, isolated network you fully control

Separate public-facing resources (web servers) from private ones (databases)

Control exactly which traffic enters and exits using security groups and NACLs

Connect your on-premises data centre to AWS securely via VPN or Direct Connect

Meet compliance requirements (PCI-DSS, HIPAA) with network-level isolation

2. Core Concepts & Terminology

Term	Definition	Example / Detail
VPC (Virtual Private Cloud)	A logically isolated virtual network in an AWS region with a defined IP address range	CIDR: 10.0.0.0/16 (65,536 IPs)
CIDR Block (Classless Inter-Domain Routing)	IP address range notation defining the VPC's address space	10.0.0.0/16 = 10.0.0.0 to 10.0.255.255
Subnet	A range of IP addresses within a VPC, tied to a single Availability Zone	10.0.1.0/24 in AZ-1 (254 usable IPs)
Public Subnet	Subnet with a route to an Internet Gateway — resources can have public IPs	Web servers, load balancers, NAT gateways

Term	Definition	Example / Detail
Private Subnet	Subnet with NO route to Internet Gateway — no direct internet access	Databases, app servers, Lambda functions
Availability Zone (AZ)	A physically separate data centre within a region. Subnets are AZ-specific	us-east-1a, us-east-1b, us-east-1c
Internet Gateway (IGW)	Horizontally scaled, highly available VPC component that enables internet access for public subnets	One IGW per VPC — attach to enable internet
NAT Gateway	Allows instances in private subnets to initiate outbound internet connections (software updates, API calls) but blocks inbound	Private EC2 pulling packages from internet
Route Table	A set of rules (routes) that determine where network traffic is directed. Every subnet has one	0.0.0.0/0 -> IGW (internet-bound traffic)
Security Group (SG)	Stateful virtual firewall at the instance/ENI level. Rules are allow-only (no explicit deny)	Allow port 443 inbound from 0.0.0.0/0
NACL (Network Access Control List)	Stateless firewall at the subnet level. Supports both allow and deny rules. Rules evaluated in order	Deny all traffic from 192.0.2.0/24 at subnet level
Elastic IP (EIP)	A static public IPv4 address you own — can be associated/reassociated with instances	Fixed public IP for a NAT Gateway or bastion
VPC Peering	Private connection between two VPCs (same or different account/region) — not transitive	Dev VPC <-> Prod VPC direct connection
Transit Gateway (TGW)	Regional hub that connects multiple VPCs, VPNs, and Direct Connect — fully transitive routing	Connect 100s of VPCs through one hub
VPN Gateway (VGW)	AWS-side endpoint for a Site-to-Site VPN connection to on-premises network	On-prem office <-> AWS VPC via IPsec VPN
Direct Connect (DX)	Dedicated private physical network connection from on-premises to AWS — not VPN	1 Gbps / 10 Gbps private line to AWS
VPC Flow Logs	Capture metadata about IP traffic going to/from network interfaces in a VPC	Log all traffic to CloudWatch/S3 for analysis
ENI (Elastic Network Interface)	A virtual network card — primary network interface for any EC2 instance	Each EC2 has at least one ENI with a private IP
Bastion Host	A hardened EC2 instance in a public subnet used as a jump server to SSH into private instances	SSH -> bastion (public) -> private EC2
VPC Endpoint	Private access to AWS services (S3, DynamoDB) without internet — gateway or interface type	S3 gateway endpoint, Secrets Manager interface endpoint

Security Group vs NACL — Key Differences

Dimension	Security Group	Network ACL (NACL)
Level	Instance / ENI level	Subnet level
State	Stateful (return traffic auto-allowed)	Stateless (must allow inbound AND outbound separately)
Rules	Allow rules only	Allow AND Deny rules
Rule Evaluation	All rules evaluated together	Rules evaluated in number order (lowest first)
Default	All outbound allowed; all inbound denied	All inbound and outbound allowed
Applies To	Specific instances within subnet	All instances in the subnet
Use For	Per-instance access control	Subnet-wide blocking (blacklisting IPs)

3. Architecture — Standard 3-Tier VPC Design

Production VPC Architecture (Multi-AZ, 3-Tier)

```
REGION: us-east-1 VPC CIDR: 10.0.0.0/16

AZ-1 (us-east-1a) AZ-2 (us-east-1b)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ INTERNET GATEWAY (IGW) ■
■ | ■
■ PUBLIC SUBNETS 10.0.1.0/24 10.0.2.0/24 ■
■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■
■ ■ ALB Target ■ ■ ALB Target ■ <- Application ■
■ ■ NAT GW ■ ■ NAT GW ■ Load Balancer ■
■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■
■ | (NAT for private subnets) ■
■ PRIVATE APP SUBNETS 10.0.11.0/24 10.0.12.0/24 ■
■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■
■ ■ EC2/ECS ■ ■ EC2/ECS ■ <- App servers ■
■ ■ Lambda ■ ■ Lambda ■ (no public IP) ■
■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■
■ ■
■ PRIVATE DATA SUBNETS 10.0.21.0/24 10.0.22.0/24 ■
■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■
■ ■ RDS Primary■ ■ RDS Standby■ <- Databases ■
■ ■ ElastiCache■ ■ ■ (no internet) ■
■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

ROUTE TABLES:
Public Subnet RT: 0.0.0.0/0 -> IGW
Private App Subnet RT: 0.0.0.0/0 -> NAT Gateway
Private Data Subnet RT: (no internet route — local only)

SECURITY:
ALB SG: Inbound 443 from 0.0.0.0/0
App SG: Inbound 8080 from ALB SG only
DB SG: Inbound 5432 from App SG only
```

4. Key Features

Feature	Description	Business Value
Custom IP Ranges	Define VPC CIDR (e.g. 10.0.0.0/16); add secondary CIDRs later	Full control over IP address space

Feature	Description	Business Value
Multiple Subnets	Create public, private, isolated subnets across AZs	Separate tiers: web, app, data
Security Groups	Stateful instance-level firewalls — allow rules only	Zero-trust per-instance access control
Network ACLs	Stateless subnet-level firewalls — allow and deny rules	Subnet-wide traffic blocking
Internet Gateway	Enable bidirectional internet access for public subnets	Host public-facing web servers
NAT Gateway	Enable outbound-only internet access from private subnets	Private servers get software updates securely
VPC Peering	Private connectivity between two VPCs without internet	Cross-VPC or cross-account communication
Transit Gateway	Regional hub for complex multi-VPC and hybrid connectivity	Simplify large multi-VPC architectures
VPN Gateway	IPsec VPN to on-premises network	Hybrid cloud with existing infrastructure
Direct Connect	Dedicated private physical connection to AWS	Consistent high-bandwidth, low-latency hybrid link
VPC Flow Logs	Log all IP traffic metadata to CloudWatch Logs or S3	Security analysis, compliance, troubleshooting
VPC Endpoints	Private access to AWS services without internet	Secure access to S3, DynamoDB, and more
IPv6 Support	Dual-stack VPC with both IPv4 and IPv6 CIDRs	Future-proof networking
Traffic Mirroring	Copy ENI traffic to monitoring appliances	Deep packet inspection, intrusion detection
Network Manager	Centralised network visibility and management across all VPCs/TGWs	Global network monitoring dashboard

5. Real-World Use Cases

Scenario	VPC Configuration	Key Components
Standard Web App	Public subnet: ALB; Private subnet: EC2/ECS; Data subnet: RDS; NAT GW for outbound	IGW, NAT GW, 3-tier subnets, SG per tier
Serverless App	Lambda in VPC private subnet accesses RDS and ElastiCache; VPC endpoints for S3/Secrets Manager	VPC Lambda, Interface Endpoints, SG
Hybrid Enterprise	Corporate DC connects via Direct Connect to VPC; internal DNS via Route 53 Resolver; all traffic private	Direct Connect, VGW, Private Hosted Zone
Multi-Account Org	Shared services VPC (logging, security, DNS) connected to workload VPCs via Transit Gateway	Transit Gateway, RAM, PrivateLink
Financial Services (PCI-DSS)	Cardholder data in isolated subnet with no internet; NACL blocks all non-essential traffic; flow logs to SIEM	Isolated subnet, strict NACLs, VPC Flow Logs
Machine Learning	Training cluster in private subnet with EFS; SageMaker in VPC mode; S3 Gateway Endpoint for data	Private subnet, EFS, S3 Gateway Endpoint
Multi-Region DR	Primary VPC in us-east-1, DR VPC in us-west-2; Route 53 failover routing; RDS cross-region replica	Multi-region VPCs, Route 53 Failover, RDS Replica

Scenario	VPC Configuration	Key Components
Zero-Trust Network	All traffic between services authenticated via security groups and IAM; no broad CIDR rules	Fine-grained SG rules referencing SG IDs

6. AWS Service Integrations

AWS Service	Integration	Purpose
Amazon EC2	All instances launch in a VPC subnet with private IPs	Compute within your private network
Amazon RDS	DB instances in private subnets; Multi-AZ across subnets in different AZs	Private, highly available databases
AWS Lambda	Lambda in VPC accesses private resources (RDS, ElastiCache)	Serverless access to private resources
Amazon EKS / ECS	Worker nodes / tasks in VPC subnets; pods get VPC IPs (EKS VPC CNI)	Container networking
AWS PrivateLink	Interface Endpoints in VPC subnets for private AWS service access	Private AWS API access
AWS Transit Gateway	Connect multiple VPCs and on-prem via a central hub	Hub-and-spoke network architecture
AWS Direct Connect	Dedicated physical line terminates at VGW in VPC	High-bandwidth private hybrid connectivity
VPC Flow Logs -> CloudWatch / S3	Stream network metadata for analysis	Security monitoring, cost analysis, debugging
AWS Network Firewall	Stateful managed firewall deployed in dedicated subnets	Advanced traffic inspection and filtering
Amazon GuardDuty	Analyses VPC Flow Logs for threat detection	Network-based threat detection
AWS Config	Tracks VPC configuration changes over time	Compliance and drift detection

7. VPC Connectivity Options Compared

Option	Connects	Transitive?	Internet?	Best For
VPC Peering	VPC <-> VPC	No (one hop only)	No	Simple VPC-to-VPC connection, same or different account/region
Transit Gateway	Many VPCs + VPN + DX	Yes (hub)	No	Large orgs with many VPCs; replaces full-mesh peering
Site-to-Site VPN	On-prem <-> VPC	Via TGW	Uses internet (encrypted)	Quick on-prem connection; lower cost than DX
Direct Connect	On-prem <-> VPC	Via TGW/DXGW	No (dedicated)	Consistent high-BW, low-latency private link to AWS
PrivateLink	VPC <-> Service	No	No	Expose service to other VPCs/accounts privately
Internet Gateway	VPC <-> Internet	N/A	Yes	Public-facing resources
NAT Gateway	Private subnet <-> Internet (outbound)	N/A	Yes (outbound only)	Private resources need software updates / external API calls

8. Chapter Summary — All 4 Services Together

Service	Layer	Primary Function	Without It...
Amazon VPC	Network Foundation	Your private network in AWS — subnets, routing, firewalls	No isolation, no control over network
AWS PrivateLink	Private Connectivity	Expose/access services privately between VPCs without internet	Must use internet or complex VPC peering for service access
Amazon Route 53	DNS & Traffic Routing	Translate domain names to IPs; route traffic intelligently	No human-readable URLs; no failover; no global routing
Amazon CloudFront	Content Delivery (CDN)	Cache and deliver content from edge locations worldwide	All traffic hits origin; slow for global users; origin overloaded

How All 4 Services Work Together — A Complete Architecture

1. USER requests https://www.myapp.com from Tokyo

2. ROUTE 53 resolves www.myapp.com

Latency routing -> returns CloudFront distribution IP (nearest edge)

3. CLOUDFRONT edge (Tokyo PoP) receives request

Static content (CSS/JS/images) -> cache HIT -> returned instantly

Dynamic API call (/api/user) -> cache MISS -> forward to origin

WAF evaluates request -> not malicious -> proceed

4. CloudFront forwards to ORIGIN: Application Load Balancer (us-east-1)

Traffic travels over AWS private backbone (not public internet)

5. ALB is in a PUBLIC SUBNET inside the VPC

Routes to EC2/ECS app servers in PRIVATE SUBNETS

6. App servers access RDS database in PRIVATE DATA SUBNET

App servers access Secrets Manager via PRIVATELINK (Interface Endpoint)

App servers access S3 via PRIVATELINK (Gateway Endpoint)

No internet route needed for any of these calls

7. RESPONSE travels back: RDS -> App -> ALB -> CloudFront edge -> User

VPC = the isolated network | PrivateLink = internal private service access

Route 53 = DNS + smart routing | CloudFront = global edge delivery + caching